Final Project, Option 3: How much structure can you find without the labels?

An agricultural co-op receives unlabelled deliveries of mixed wheat grain and wants to know how many distinct varieties are present, and how cleanly they separate, before committing to any sorting process. In this project you recover that structure using clustering alone.

In this project you take a data set with a known grouping, hide that grouping from yourself, and see how much of it you can recover using clustering alone. The labels are kept aside until the very end, where they serve only as an answer key.

Central question: how much of the real class structure can you recover without ever using the labels?

The data set is Seeds: 210 wheat kernels described by seven geometric measurements such as area, perimeter, and kernel length. You are given the measurements to work with, and the variety labels are set aside for the final check.

This project draws on Module 4. There we scaled features, reduced them with PCA, clustered with K-Means, chose the number of clusters using the elbow and silhouette methods, and validated the result against a known label with a cross-tabulation. Here you carry out that whole process yourself and judge how faithfully the clusters match the real varieties.

How this notebook is organised. The cells in Part A are already written for you. They handle the import, the download, and the scaling. From Part B onward, the cells are yours to complete. Each one carries comments describing what to do, along with a few hints, but not the finished code. Work through them in order, and resist looking at the varieties until Part D.

Student: Andreou Maria-Gavriela

> **Summary of my results** *(written after finishing the notebook, so a reader knows where it ends up)*
>
> - **Number of groups:** the elbow pointed to **k = 3** and the silhouette to **k = 2**. I used **k = 3**, because the gap statistic also selected 3, and bootstrap refits showed three clusters are about as reproducible as two, while four or more are not.
> - **Agreement with the real varieties: purity 0.919**, meaning 193 of 210 kernels sit in a cluster dominated by their own variety. The chance-corrected scores agree: **adjusted Rand index 0.773** and **NMI 0.728**.
> - **Cleanest and hardest varieties:** Rosa (the large kernels) separates most cleanly. Kama sits between the other two, so **Kama and Canadian** are the hardest pair to tell apart (10 kernels swapped), followed by Kama and Rosa (7). Rosa and Canadian are never confused.
> - **How close to the best possible:** a supervised model that is *allowed* to see the labels gets 96.2% of unseen kernels right; clustering gets 91.9% without ever seeing them.
> - **How much of the structure is real:** most of it. Two other algorithms find much the same three groups, and K-Means gives the identical answer from 25 different random starts. The boundaries are soft, though: most of the 17 misplaced kernels are borderline cases that sit almost exactly between two cluster centres.
>
> Cells marked *extension* go beyond the required steps. They never change the clustering, and none of them looks at the labels before Part D.

Part A. Setup and data

We start with the imports. This cell loads everything the notebook needs, so we run it once at the top.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

A few extra imports and settings that I use in my own cells: the metrics and models for the extension analyses, a consistent plotting style, and a `figures/` folder where every chart is also saved for the README.

In [2]:
```
from pathlib import Path
from scipy.optimize import linear_sum_assignment          # used in Part D only
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score

RANDOM_STATE = 42                     # one fixed seed, so every run gives the same output

FIG_DIR = Path("figures")
FIG_DIR.mkdir(exist_ok=True)

plt.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 150, "savefig.bbox": "tight",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.25, "font.size": 10.5,
})

# One colour per variety, used only in Part D once the labels are revealed.
VARIETY_COLOURS = {1: "#2a9d8f", 2: "#e9c46a", 3: "#7b5ea7"}
```

Loading the data

We download the Seeds data set. The seven measurements go into `X`, and the true varieties go into `y_true`. We put the varieties aside now and do not touch them until Part D, so that the clustering is done with no knowledge of the answer.

In [3]:
```
# The Seeds data set: 210 kernels described by seven geometric measurements.
# It is small and fully numeric,
# which suits clustering well.
from pathlib import Path

feature_names = ["area", "perimeter", "compactness", "kernel_length",
                 "kernel_width", "asymmetry", "groove_length"]

# A copy of the same UCI Seeds file is kept in data/seeds.csv so the notebook also runs
# offline. If that file is not next to the notebook, the data are downloaded from
# OpenML exactly as in the course version of this cell.
LOCAL_COPY = Path("data/seeds.csv")
if LOCAL_COPY.exists():
    raw = pd.read_csv(LOCAL_COPY, header=None)      # 7 measurements + variety code
    X = raw.iloc[:, :7].copy()
    y_true = raw.iloc[:, 7].to_numpy().astype(int)
    source = f"{LOCAL_COPY} (copy of the UCI Seeds file)"
else:
    seeds = fetch_openml(name="seeds", version=1, as_frame=True)
    X = seeds.data.copy()
    y_true = seeds.target.to_numpy().astype(int)
    source = "OpenML (seeds, version 1)"

X.columns = feature_names          # give the seven measurements readable names
```

```
# The true varieties are now in y_true. We set them aside and do NOT use them until
# Part D, where they serve only as an external check on the clustering.
assert X.shape == (210, 7) and X.notna().all().all()

print("Data source:", source)
print("Rows and features:", X.shape)
X.head()
```

```
Data source: data/seeds.csv (copy of the UCI Seeds file)
Rows and features: (210, 7)
```

Out[3]:

	area	perimeter	compactness	kernel_length	kernel_width	asymmetry	groove_length
0	15.26	14.84	0.8710	5.763	3.312	2.221	5.220
1	14.88	14.57	0.8811	5.554	3.333	1.018	4.956
2	14.29	14.09	0.9050	5.291	3.337	2.699	4.825
3	13.84	13.94	0.8955	5.324	3.379	2.259	4.805
4	16.14	14.99	0.9034	5.658	3.562	1.355	5.175

Scaling the features

Clustering relies on distances between points, so a feature measured on a larger scale would count for more than the others. We put every feature on the same footing with `StandardScaler`, as in Module 4.

In [4]:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Scaled feature matrix:", X_scaled.shape)
```

```
Scaled feature matrix: (210, 7)
```

Part B. Explore, without labels

From here on, the cells are yours to complete.

First, look at the shape of the data. Seven measurements are too many to plot at once, so reduce them to two with PCA and draw a scatter. This is the same reduction you used in Module 4.

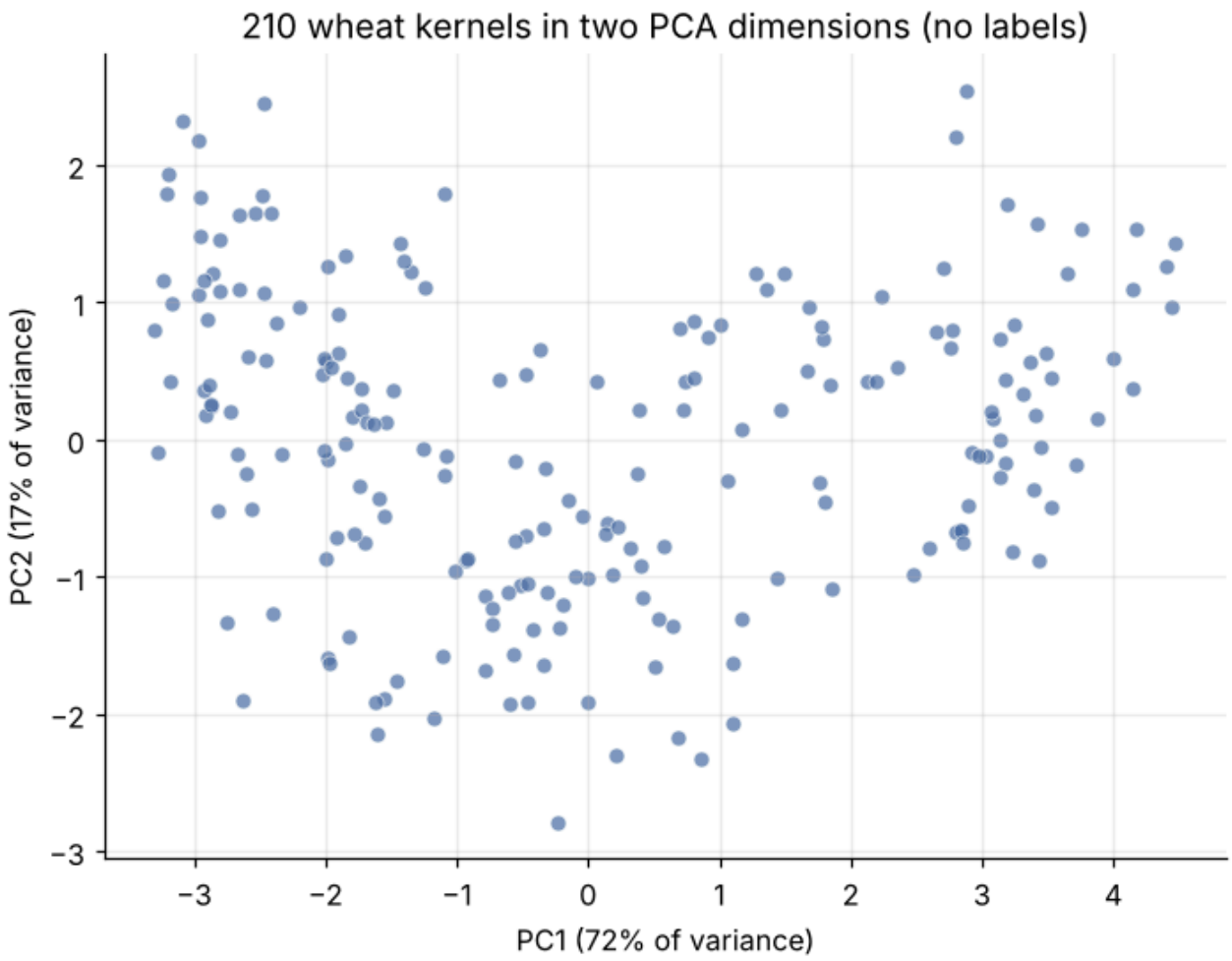
In [5]:

```
# Reduce the seven scaled features to two principal components so the data can be drawn.
pca = PCA(n_components=2)
X2 = pca.fit_transform(X_scaled)

evr = pca.explained_variance_ratio_
print("Explained variance ratio (PC1, PC2):", evr.round(3))
print(f"Variation captured by the two components together: {evr.sum():.1%}")

# A single colour on purpose: here we study the shape of the data, not any labels.
fig, ax = plt.subplots(figsize=(6.6, 5.2))
ax.scatter(X2[:, 0], X2[:, 1], s=34, color="#4a6fa5", alpha=0.7,
           edgecolor="white", linewidth=0.5)
ax.set_xlabel(f"PC1 ({evr[0]:.0%} of variance)")
ax.set_ylabel(f"PC2 ({evr[1]:.0%} of variance)")
ax.set_title("210 wheat kernels in two PCA dimensions (no labels)")
fig.tight_layout()
fig.savefig(FIG_DIR / "01_pca_unlabelled.png")
plt.show()
```

```
Explained variance ratio (PC1, PC2): [0.719 0.171]
Variation captured by the two components together: 89.0%
```



Going further: what do the two PCA axes measure? *(extension, still no labels)*

Before reading too much into the scatter, I check what its two axes mean. The loadings show how much each scaled measurement contributes to each component. The correlation matrix shows why two components can capture almost 90% of the variation.

In [6]:

```
# What do the two axes mean? Loadings = weight of each scaled feature in each component.
loadings = pd.DataFrame(pca.components_.T, index=feature_names, columns=["PC1", "PC2"])
print("Variance explained by all seven components:",
      PCA().fit(X_scaled).explained_variance_ratio_.round(3))
display(loadings.round(2))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12.5, 4.8),
                               gridspec_kw={"width_ratios": [1.1, 1]})

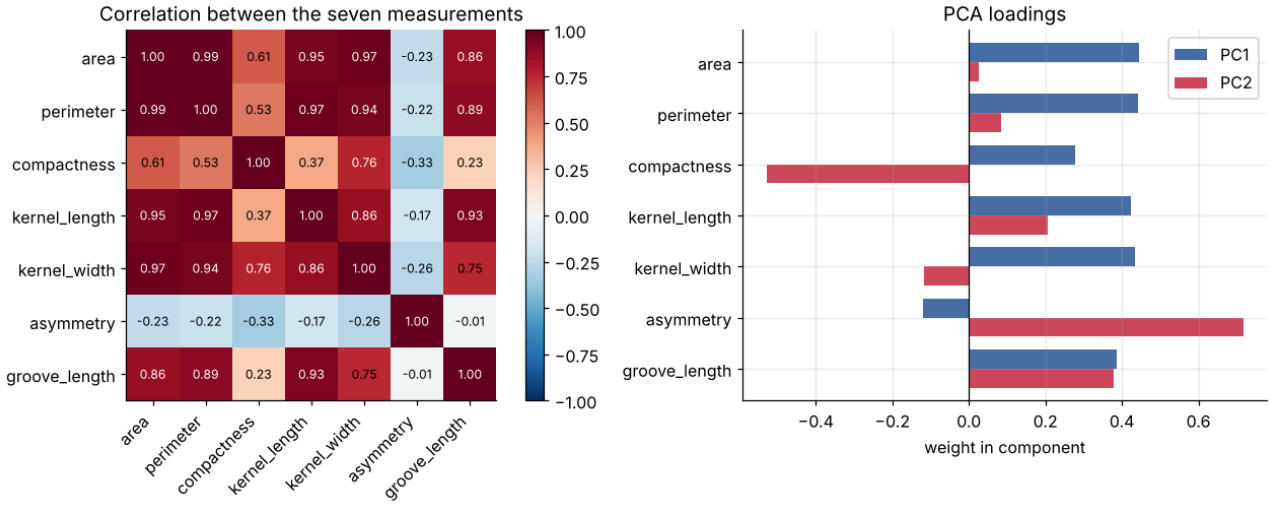
corr = X.corr()
im = ax1.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
for i in range(len(corr)):
    for j in range(len(corr)):
        ax1.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", fontsize=8,
                 color="white" if abs(corr.iloc[i, j]) > 0.75 else "black")
ax1.set_xticks(range(7), feature_names, rotation=45, ha="right")
ax1.set_yticks(range(7), feature_names)
ax1.set_title("Correlation between the seven measurements")
ax1.grid(False)
fig.colorbar(im, ax=ax1, fraction=0.046)

loadings.plot.barh(ax=ax2, color=["#4a6fa5", "#d1495b"], width=0.75)
ax2.axvline(0, color="black", lw=0.8)
ax2.invert_yaxis()
ax2.set_title("PCA loadings")
ax2.set_xlabel("weight in component")
```

```
fig.tight_layout()
fig.savefig(FIG_DIR / "05_correlation_loadings.png")
plt.show()
```

```
Variance explained by all seven components: [0.719 0.171 0.097 0.01  0.003 0.001 0.   ]
```

	PC1	PC2
area	0.44	0.03
perimeter	0.44	0.08
compactness	0.28	-0.53
kernel_length	0.42	0.21
kernel_width	0.43	-0.12
asymmetry	-0.12	0.72
groove_length	0.39	0.38



What this shows.

- **PC1 (72%) is a size axis.** Area, perimeter, kernel length, kernel width and groove length all load almost equally on it (0.39–0.44). That makes sense, because these five measurements are very strongly correlated with each other (r = 0.75–0.99).
- **PC2 (17%) is a shape axis.** It contrasts asymmetry (+0.72) and groove length (+0.38) with compactness (−0.53).

So in the scatter, small kernels are on the left and large kernels on the right, and kernels near the top are more asymmetric and less compact. Since five of the seven measurements mostly repeat the same size information, size will dominate any distance-based clustering. That is worth remembering when I interpret the clusters later.

What do you see?

The kernels form a curved band rather than clean blobs, with three loose concentrations (a dense group on the left, one on the right, and a more spread-out group lower in the middle), so I expect **about three groups**. They are not cleanly separated: the left and middle groups run into each other around PC1 ≈ −2 to −1, and the middle group trails into the right one around PC1 ≈ 1 to 2, while the right-hand group looks the most distinct.

Part C. Cluster and choose the number of groups

Now decide how many clusters the data supports, using the two methods from Module 4. The elbow method looks at inertia, the total spread of points around their cluster centres, and finds where adding

another cluster stops helping much. The silhouette score measures how well separated the clusters are, where higher is better.

In [7]:
```
# For each k from 2 to 7, fit K-Means and record the inertia and the silhouette score.
k_values = list(range(2, 8))
inertias, silhouettes, sizes = [], [], []

for k in k_values:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)                                # elbow method
    silhouettes.append(silhouette_score(X_scaled, km.labels_))  # separation
    sizes.append(sorted(np.bincount(km.labels_).tolist(), reverse=True))

diagnostics = pd.DataFrame({"k": k_values,
                            "inertia": inertias,
                            "drop in inertia from k-1": -pd.Series(inertias).diff(),
                            "silhouette": silhouettes,
                            "cluster sizes": sizes}).set_index("k")
display(diagnostics.round(3))

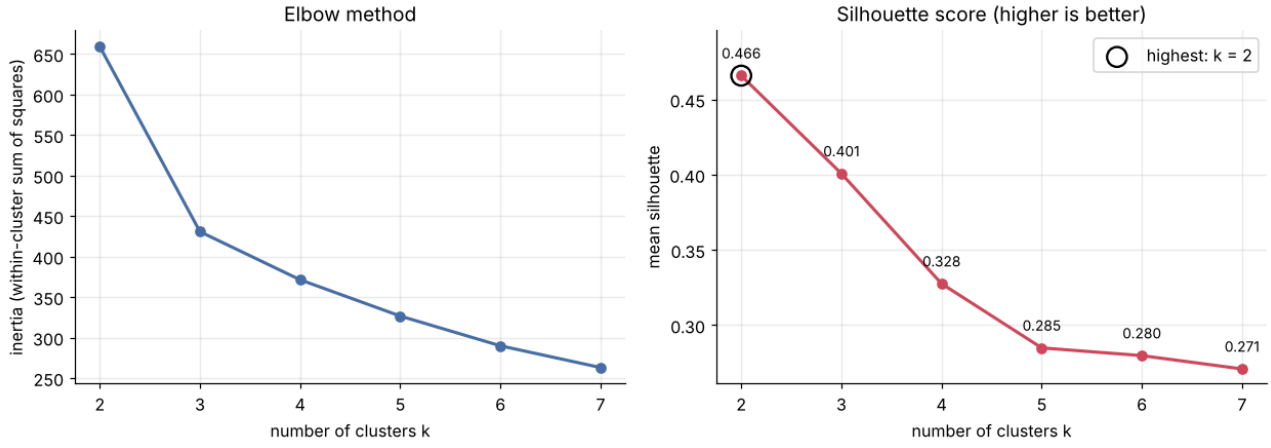
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.2))

ax1.plot(k_values, inertias, "o-", color="#4a6fa5", lw=2)
ax1.set(title="Elbow method", xlabel="number of clusters k",
        ylabel="inertia (within-cluster sum of squares)", xticks=k_values)

best_sil = k_values[int(np.argmax(silhouettes))]
ax2.plot(k_values, silhouettes, "o-", color="#d1495b", lw=2)
ax2.scatter([best_sil], [max(silhouettes)], s=160, facecolor="none",
            edgecolor="black", lw=1.5, zorder=3, label=f"highest: k = {best_sil}")
for kk, s in zip(k_values, silhouettes):
    ax2.annotate(f"{s:.3f}", (kk, s), textcoords="offset points", xytext=(0, 11),
                 ha="center", fontsize=9)
ax2.set(title="Silhouette score (higher is better)", xlabel="number of clusters k",
        ylabel="mean silhouette", xticks=k_values)
ax2.legend(loc="upper right")
ax2.set_ylim(top=max(silhouettes) + 0.03)

fig.tight_layout()
fig.savefig(FIG_DIR / "02_elbow_silhouette.png")
plt.show()
```

k	inertia	drop in inertia from k-1	silhouette	cluster sizes
2	659.172	NaN	0.466	[133, 77]
3	430.659	228.513	0.401	[72, 71, 67]
4	371.302	59.357	0.328	[67, 64, 49, 30]
5	326.514	44.788	0.285	[58, 48, 39, 36, 29]
6	289.804	36.710	0.280	[48, 45, 35, 34, 27, 21]
7	262.974	26.830	0.271	[51, 34, 30, 28, 26, 22, 19]



Going further: two more label-free checks on k *(extension)*

The elbow and the silhouette disagree, so before deciding I add two independent checks. Neither one uses the labels.

- **Gap statistic** (Tibshirani, Walther & Hastie, 2001). It compares the inertia of the real data with the inertia of reference data sets that have no clusters at all. The rule picks the smallest k whose gap is at least as large as the next one, minus one standard error.
- **Bootstrap stability.** If a clustering is real, refitting it on resampled data should split the kernels the same way. I measure that agreement with the adjusted Rand index (ARI). Here the ARI compares two clusterings with each other, never a clustering with the varieties.

In [8]:

```
rng = np.random.default_rng(RANDOM_STATE)

def kmeans_inertia(data, n_clusters):
    model = KMeans(n_clusters=n_clusters, n_init=10, random_state=RANDOM_STATE)
    return model.fit(data).inertia_

# 1) Gap statistic (Tibshirani, Walther and Hastie, 2001).
#    Reference data sets have no clusters: points spread uniformly over the box that
#    the data occupy along their principal axes. A real cluster structure makes the
#    observed log-inertia fall clearly below the reference one.
B = 20
_, _, Vt = np.linalg.svd(X_scaled - X_scaled.mean(axis=0), full_matrices=False)
X_rot = X_scaled @ Vt.T
k_gap = list(range(1, 8))
gap, gap_se = [], []
for kk in k_gap:
    ref_logs = []
    for _ in range(B):
        reference = rng.uniform(X_rot.min(0), X_rot.max(0), X_rot.shape) @ Vt
        ref_logs.append(np.log(kmeans_inertia(reference, kk)))
    gap.append(np.mean(ref_logs) - np.log(kmeans_inertia(X_scaled, kk)))
    gap_se.append(np.std(ref_logs) * np.sqrt(1 + 1 / B))
# Tibshirani's rule: the smallest k with Gap(k) >= Gap(k+1) - s(k+1)
gap_choice = next(kk for i, kk in enumerate(k_gap[:-1])
                  if gap[i] >= gap[i + 1] - gap_se[i + 1])

# 2) Bootstrap stability: refit K-Means on resampled data and check, with the adjusted
#    Rand index (ARI), whether it splits the full data set the same way as before.
#    ARI = 1 means an identical partition; ARI near 0 means agreement no better than chance.
n_boot = 50
stability = {}
for kk in range(2, 8):
    ref_model = KMeans(n_clusters=kk, n_init=10, random_state=RANDOM_STATE)
    ref_labels = ref_model.fit_predict(X_scaled)
    scores = []
    for b in range(n_boot):
        idx = rng.integers(0, len(X_scaled), len(X_scaled))
        boot_model = KMeans(n_clusters=kk, n_init=10, random_state=b).fit(X_scaled[idx])
        scores.append(adjusted_rand_score(ref_labels, boot_model.predict(X_scaled)))
    stability[kk] = scores

boot_mean = {kk: np.mean(v) for kk, v in stability.items()}
boot_p5 = {kk: np.percentile(v, 5) for kk, v in stability.items()}
summary = pd.DataFrame({"gap": pd.Series(gap, index=k_gap),
                        "gap s.e.": pd.Series(gap_se, index=k_gap),
                        "bootstrap ARI (mean)": pd.Series(boot_mean),
                        "bootstrap ARI (5th pct)": pd.Series(boot_p5)}).round(3)
summary.index.name = "k"
display(summary)
print("Gap statistic rule selects k =", gap_choice)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.2))
ax1.errorbar(k_gap, gap, yerr=gap_se, fmt="o-", color="#4a6fa5", lw=2, capsize=4)
ax1.scatter([gap_choice], [gap[k_gap.index(gap_choice)]], s=170, facecolor="none",
            edgecolor="black", lw=1.5, zorder=3, label=f"rule selects k = {gap_choice}")
ax1.set(title="Gap statistic", xlabel="number of clusters k", ylabel="gap", xticks=k_gap)
ax1.legend(loc="lower right")

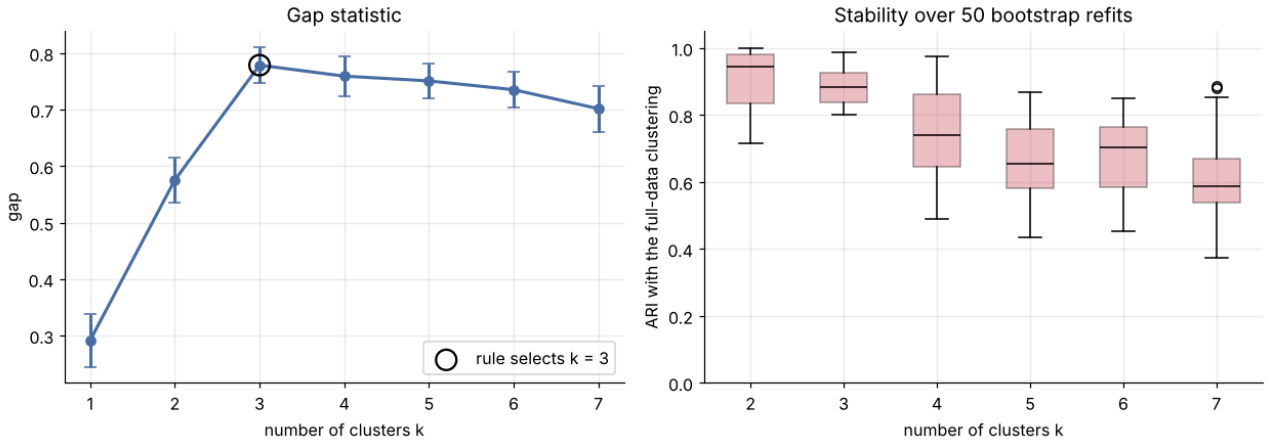
ax2.boxplot([stability[kk] for kk in range(2, 8)], positions=list(range(2, 8)), widths=0.5,
```

```
            patch_artist=True, boxprops=dict(facecolor="#d1495b", alpha=0.35),
            medianprops=dict(color="black"))
ax2.set(title=f"Stability over {n_boot} bootstrap refits", xlabel="number of clusters k",
        ylabel="ARI with the full-data clustering", ylim=(0, 1.05))

fig.tight_layout()
fig.savefig(FIG_DIR / "06_gap_stability.png")
plt.show()
```

k	gap	gap s.e.	bootstrap ARI (mean)	bootstrap ARI (5th pct)
1	0.292	0.047	NaN	NaN
2	0.575	0.040	0.920	0.774
3	0.779	0.032	0.879	0.800
4	0.759	0.035	0.745	0.518
5	0.751	0.031	0.663	0.461
6	0.735	0.032	0.673	0.481
7	0.702	0.040	0.611	0.435

```
Gap statistic rule selects k = 3
```



What this shows.

- **Gap statistic: the rule selects k = 3.** The gap rises steeply up to 3 (0.29 → 0.58 → 0.78), then flattens and slowly declines. Adding a fourth cluster no longer finds structure that random, cluster-free data would not also give.
- **Bootstrap stability: k = 2 and k = 3 are both stable.** Their mean ARIs are 0.92 and 0.88, and k = 3 has the tighter spread (5th percentile 0.80 against 0.77). From k = 4 onwards the partitions start to change from one resample to the next (mean ARI 0.75 or lower, 5th percentile about 0.52 or lower). This suggests that the extra clusters cut through continuous regions instead of finding new groups.

Taken together, the elbow and the gap statistic point to k = 3 and the silhouette to k = 2. Stability rules out k ≥ 4 but cannot choose between 2 and 3.

How many clusters?

I use k = 3. The elbow is sharp at k = 3: inertia falls by 229 from k = 2 to 3, but only by 59 from 3 to 4. The silhouette is highest at k = 2 (0.466 against 0.401), but the k = 2 solution only splits the band into a 133-kernel group and a 77-kernel group, merging the left and middle concentrations that are visible in the PCA plot.

The two extra label-free checks above support this choice. The gap statistic selects k = 3. Bootstrap refits show that the k = 3 partition is about as reproducible as k = 2: mean ARI 0.88 against 0.92, with

a tighter 5th percentile (0.80 against 0.77). From k = 4 onwards, the clusters change from sample to sample.

Fit K-Means

Fit K-Means at the number of clusters you decided on, and keep the cluster assignment for each kernel.

In [9]:
```
# Fit K-Means at the chosen number of clusters and keep each kernel's assignment.
k = 3
km = KMeans(n_clusters=k, n_init=10, random_state=42)
km.fit(X_scaled)
labels = km.labels_

print(f"K-Means with k = {k}")
print("Kernels per cluster:", {c: int(n) for c, n in enumerate(np.bincount(labels))})
print(f"Inertia: {km.inertia_:.1f}   Silhouette: {silhouette_score(X_scaled, labels):.3f}")
```

```
K-Means with k = 3
Kernels per cluster: {0: 72, 1: 67, 2: 71}
Inertia: 430.7   Silhouette: 0.401
```

Part D. Judge against the labels

We now bring in the true varieties for the first time, only as an external check. The clustering itself is finished; nothing below changes it. We simply ask how well the groups you found line up with the real varieties, exactly as we validated the clusters in Module 4.

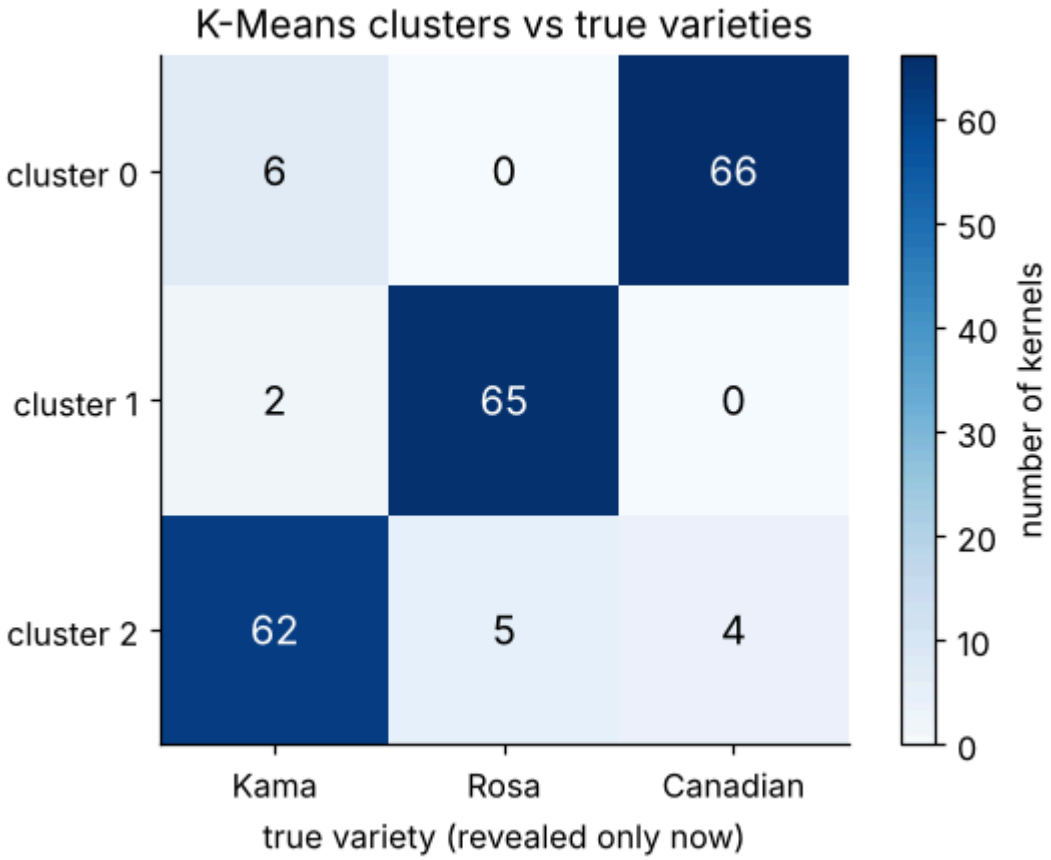
In [10]:
```
# Reveal the varieties only now, after the clustering is complete.
print("Varieties present:", np.unique(y_true))

# Variety codes used in the UCI Seeds data set.
variety_names = {1: "Kama", 2: "Rosa", 3: "Canadian"}

# Cross-tabulate the cluster labels against the true varieties.
# Each row is a cluster, each column a variety.
ct = pd.crosstab(labels, y_true, rownames=["cluster"], colnames=["true variety"])
print(ct)

# The same table as a heatmap (the clustering equivalent of a confusion matrix).
ct_named = ct.rename(columns=variety_names)
fig, ax = plt.subplots(figsize=(5.6, 4.2))
im = ax.imshow(ct_named.values, cmap="Blues")
for i in range(ct_named.shape[0]):
    for j in range(ct_named.shape[1]):
        v = ct_named.values[i, j]
        ax.text(j, i, v, ha="center", va="center", fontsize=13,
                color="white" if v > ct_named.values.max() / 2 else "black")
ax.set_xticks(range(3), ct_named.columns)
ax.set_yticks(range(k), [f"cluster {c}" for c in ct_named.index])
ax.set_xlabel("true variety (revealed only now)")
ax.set_title("K-Means clusters vs true varieties")
ax.grid(False)
fig.colorbar(im, ax=ax, label="number of kernels")
fig.tight_layout()
fig.savefig(FIG_DIR / "03_crosstab_heatmap.png")
plt.show()
```

```
Varieties present: [1 2 3]
true variety   1   2   3
cluster
0              6   0  66
1              2  65   0
2             62   5   4
```



Measuring the agreement

Turn the table into a single number with a purity score. For each cluster, take the count of its most common variety; add those counts across all clusters; then divide by the total number of kernels. A value near 1 means each cluster is almost entirely one variety.

Purity tends to increase when more clusters are created, so interpret it together with your chosen value of `k` and the cross-tabulation rather than using it on its own.

In [11]:
```
# Purity: for each cluster take the count of its most common variety,
# add these over all clusters, and divide by the total number of kernels.
majority_counts = ct.max(axis=1)
purity = majority_counts.sum() / ct.values.sum()

print("Majority count per cluster:", majority_counts.to_dict())
print(f"Purity = {majority_counts.sum()} / {ct.values.sum()} = {purity:.3f}")
```

```
Majority count per cluster: {0: 66, 1: 65, 2: 62}
Purity = 193 / 210 = 0.919
```

Going further: purity alongside chance-corrected scores *(extension)*

Purity has two weaknesses:

- It gives a single number for the whole data set and says nothing about individual varieties.
- Splitting a cluster can never lower it, so it climbs towards 1.0 as k grows and reaches exactly 1.0 when every kernel is its own cluster.

So I add a per-variety breakdown and two scores that correct for this. The **adjusted Rand index (ARI)** is 0 for a random assignment and 1 for a perfect match. **Normalised mutual information (NMI)** measures how much knowing the cluster tells us about the variety. The labels are only used here, after the clustering was fixed.

In [12]:
```
# Per-variety view: each cluster is matched to its majority variety
# (here the match is one-to-one, so it is a proper relabelling).
cluster_to_variety = ct.idxmax(axis=1)
assert cluster_to_variety.is_unique, "two clusters share a majority variety"
assigned = cluster_to_variety.loc[labels].to_numpy()       # variety each kernel is grouped with

per_variety = pd.DataFrame({
    "kernels": pd.Series(y_true).value_counts().sort_index(),
    "recovered in its cluster": [(assigned[y_true == v] == v).sum() for v in (1, 2, 3)],
    "cluster size": [(assigned == v).sum() for v in (1, 2, 3)],
}, index=[1, 2, 3])
per_variety["recall"] = per_variety["recovered in its cluster"] / per_variety["kernels"]
per_variety["precision"] = per_variety["recovered in its cluster"] / per_variety["cluster size"]
per_variety.index = [variety_names[v] for v in per_variety.index]
display(per_variety.round(3))

print(f"Adjusted Rand index (ARI): {adjusted_rand_score(y_true, labels):.3f}   "
      f"(0 = chance-level agreement, 1 = perfect)")
print(f"Normalised mutual information (NMI): {normalized_mutual_info_score(y_true, labels):.3f}")

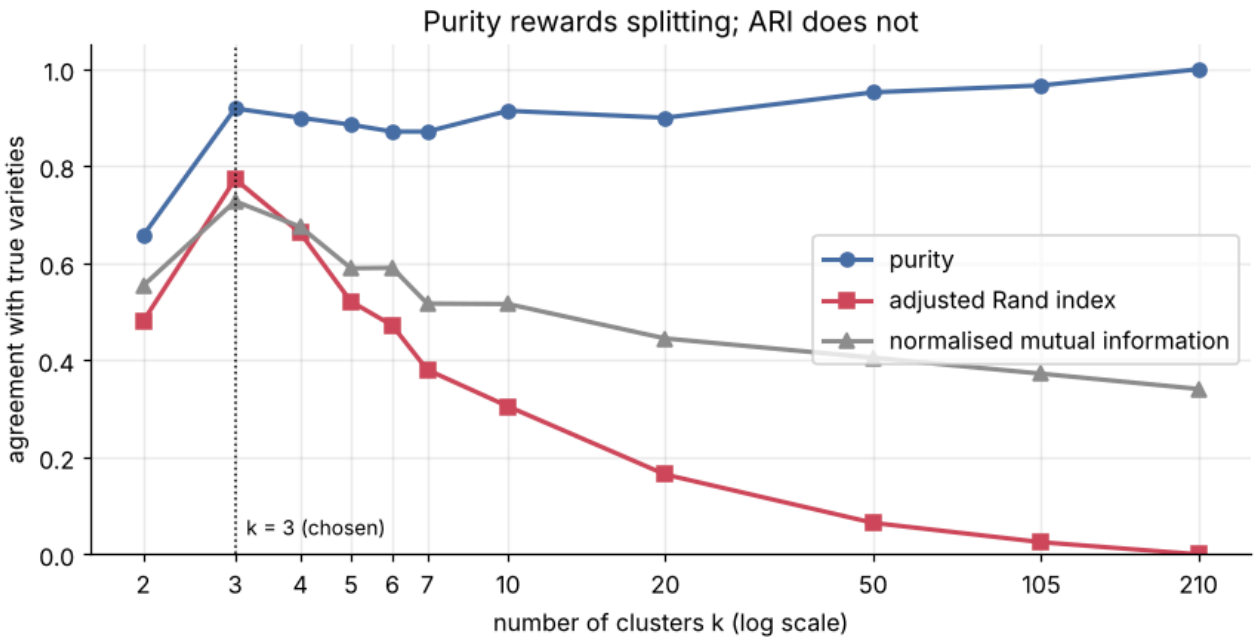
# How purity, ARI and NMI behave as k grows, up to one cluster per kernel.
rows = []
for kk in [2, 3, 4, 5, 6, 7, 10, 20, 50, 105, 210]:
    if kk == len(X_scaled):
        lab_k = np.arange(len(X_scaled))                   # every kernel in its own cluster
    else:
        model_k = KMeans(n_clusters=kk, n_init=10, random_state=RANDOM_STATE)
        lab_k = model_k.fit_predict(X_scaled)
    ct_k = pd.crosstab(lab_k, y_true)
    rows.append({"k": kk, "purity": ct_k.max(axis=1).sum() / ct_k.values.sum(),
                 "ARI": adjusted_rand_score(y_true, lab_k),
                 "NMI": normalized_mutual_info_score(y_true, lab_k)})
by_k = pd.DataFrame(rows).set_index("k")
display(by_k.round(3).T)

fig, ax = plt.subplots(figsize=(8, 4.2))
ax.plot(by_k.index, by_k["purity"], "o-", lw=2, color="#4a6fa5", label="purity")
ax.plot(by_k.index, by_k["ARI"], "s-", lw=2, color="#d1495b", label="adjusted Rand index")
ax.plot(by_k.index, by_k["NMI"], "^-", lw=2, color="#8c8c8c",
        label="normalised mutual information")
ax.axvline(3, color="black", ls=":", lw=1)
ax.text(3.15, 0.04, "k = 3 (chosen)", fontsize=9)
ax.set_xscale("log")
ax.set_xticks(by_k.index, [str(i) for i in by_k.index])
ax.minorticks_off()
ax.set(xlabel="number of clusters k (log scale)", ylabel="agreement with true varieties",
       ylim=(0, 1.05), title="Purity rewards splitting; ARI does not")
ax.legend(loc="center right")
fig.tight_layout()
fig.savefig(FIG_DIR / "07_purity_vs_k.png")
plt.show()
```

	kernels	recovered in its cluster	cluster size	recall	precision
Kama	70	62	71	0.886	0.873
Rosa	70	65	67	0.929	0.970
Canadian	70	66	72	0.943	0.917

```
Adjusted Rand index (ARI): 0.773   (0 = chance-level agreement, 1 = perfect)
Normalised mutual information (NMI): 0.728
```

k	2	3	4	5	6	7	10	20	50	105	210
purity	0.657	0.919	0.900	0.886	0.871	0.871	0.914	0.900	0.952	0.967	1.000
ARI	0.481	0.773	0.662	0.521	0.472	0.380	0.305	0.165	0.065	0.025	0.000
NMI	0.554	0.728	0.675	0.590	0.591	0.517	0.516	0.445	0.405	0.372	0.341



What this shows.

- **Rosa is recovered best:** 65 of 70 kernels, and its cluster is 97% Rosa.
- **Canadian comes next:** 66 of 70 kernels, in a cluster that is 92% Canadian.
- **Kama is the weakest:** 62 of 70 kernels, in a cluster that is 87% Kama. Kama kernels are pulled away from it on both sides.

ARI = 0.773 and NMI = 0.728 confirm that the agreement is far above chance.

The sweep over k shows the trap in purity:

- **Purity rewards splitting.** Its value at k = 3 (0.919) is beaten at k = 50 (0.952) and k = 105 (0.967), and it reaches 1.000 when every kernel is its own cluster.
- **ARI does not.** It is highest at k = 3 (0.773) and falls to 0 at k = 210.
- **The curve is not perfectly smooth.** Purity dips at k = 4 (0.900) because K-Means re-partitions the data at each k instead of splitting the existing clusters.

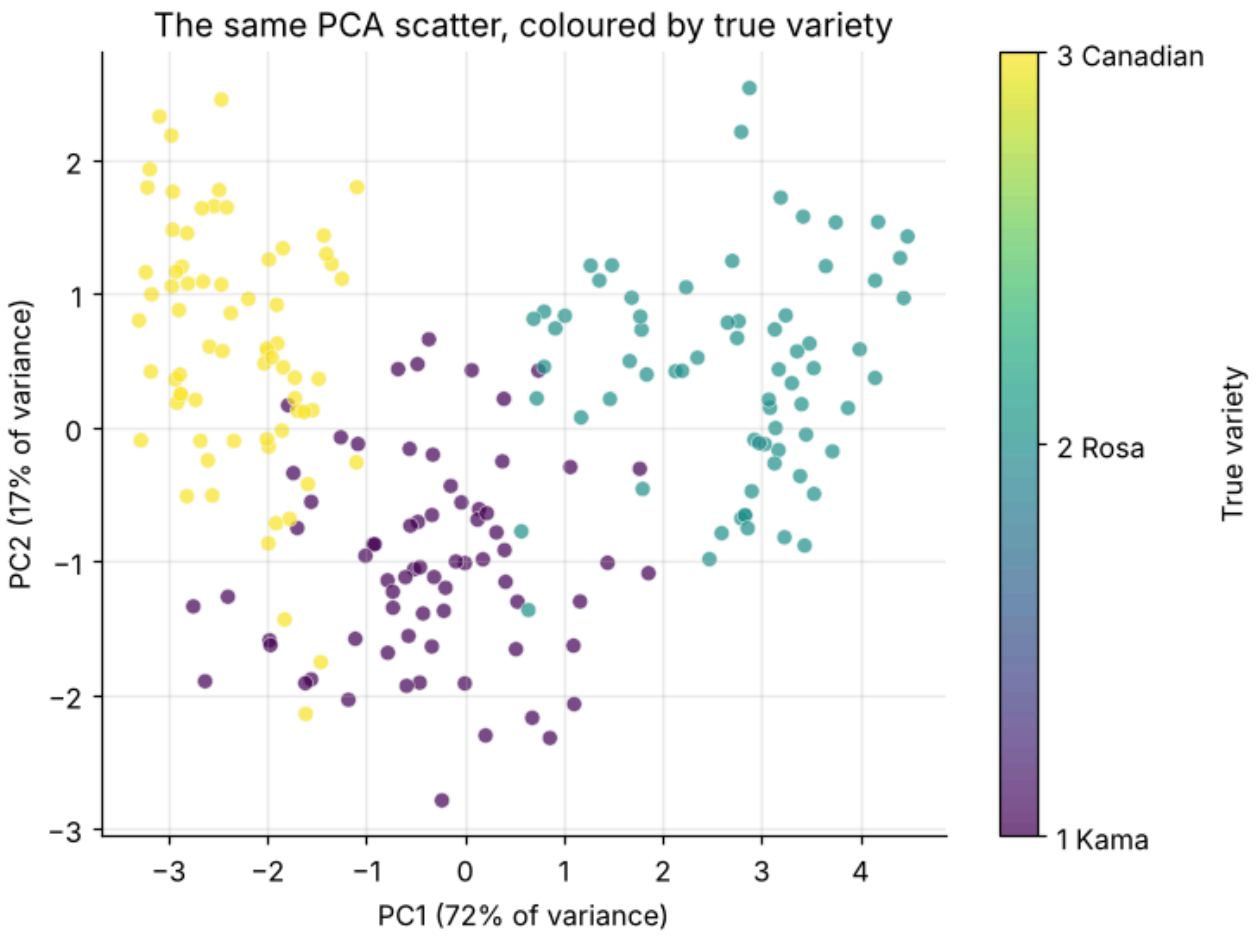
So a high purity cannot justify a choice of k on its own. The ARI peak at k = 3 is a reassuring check after the fact, but I chose k before looking at it.

Back to the picture

Connect the numbers to the shape you saw earlier. Redraw the PCA scatter, but this time colour each point by its true variety. Interpret this picture cautiously: K-Means used all seven scaled features, while the plot shows only two PCA dimensions. Points that overlap here may still be separated by information in the other dimensions.

In [13]:
```
# Redraw the PCA scatter from Part B, now coloured by the true variety.
fig, ax = plt.subplots(figsize=(6.9, 5.2))
points = ax.scatter(X2[:, 0], X2[:, 1], c=y_true, cmap="viridis", alpha=0.7,
                    s=34, edgecolor="white", linewidth=0.4)
ax.set_xlabel(f"PC1 ({evr[0]:.0%} of variance)")
ax.set_ylabel(f"PC2 ({evr[1]:.0%} of variance)")
ax.set_title("The same PCA scatter, coloured by true variety")
cbar = fig.colorbar(points, ax=ax, ticks=[1, 2, 3])
cbar.set_label("True variety")
cbar.ax.set_yticklabels([f"{c} {variety_names[c]}" for c in (1, 2, 3)])
fig.tight_layout()
```

```
fig.savefig(FIG_DIR / "04_pca_true_varieties.png")
plt.show()
```



Going further: which kernels does the clustering get wrong? *(extension)*

To look at the 17 disagreements as individual examples, I name each cluster after its majority variety and mark every kernel whose true variety is different.

For each kernel I also compute a **margin**: the distance to its second-closest cluster centre minus the distance to its closest one, in scaled units. A small margin means the kernel sits almost on the boundary between two clusters.

In [14]:
```
# Which kernels end up with a different variety from their cluster, and why?
disagree = assigned != y_true
dist = km.transform(X_scaled)                    # distance from each kernel to each centre
d_sorted = np.sort(dist, axis=1)
margin = d_sorted[:, 1] - d_sorted[:, 0]         # gap between nearest and second-nearest centre

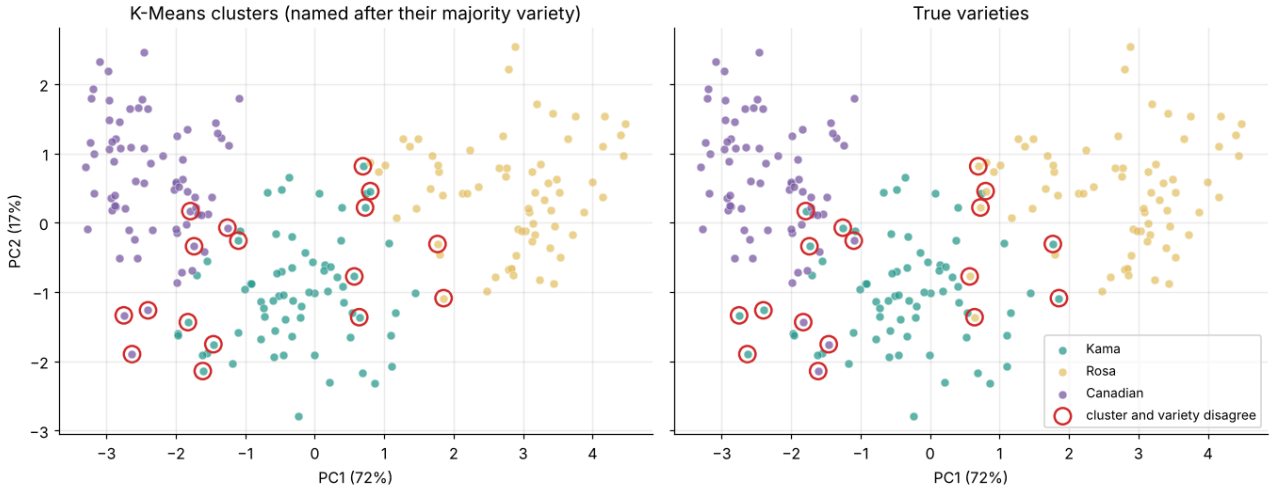
print(f"Kernels whose cluster majority differs from their true variety: "
      f"{disagree.sum()} of {len(y_true)}")
print(f"Median margin, agreeing kernels:    {np.median(margin[~disagree]):.2f}")
print(f"Median margin, disagreeing kernels: {np.median(margin[disagree]):.2f}")
low = margin <= np.quantile(margin, 0.15)
print(f"The 15% of kernels with the smallest margin ({low.sum()} kernels) contain "
      f"{(disagree & low).sum()} of the {disagree.sum()} disagreements.")

fig, axes = plt.subplots(1, 2, figsize=(12.5, 5), sharex=True, sharey=True)
panels = [(axes[0], assigned, "K-Means clusters (named after their majority variety)"),
          (axes[1], y_true, "True varieties")]
for ax, colour_by, title in panels:
    for v in (1, 2, 3):
        m = colour_by == v
        ax.scatter(X2[m, 0], X2[m, 1], s=34, alpha=0.75, color=VARIETY_COLOURS[v],
                   edgecolor="white", linewidth=0.4, label=variety_names[v])
```

```
    ax.scatter(X2[disagree, 0], X2[disagree, 1], s=130, facecolor="none",
               edgecolor="#d62828", lw=1.6, label="cluster and variety disagree")
    ax.set_title(title)
    ax.set_xlabel(f"PC1 ({evr[0]:.0%})")
axes[0].set_ylabel(f"PC2 ({evr[1]:.0%})")
axes[1].legend(loc="lower right", fontsize=9)
fig.tight_layout()
fig.savefig(FIG_DIR / "08_clusters_vs_truth.png")
plt.show()

# Example "predictions": the kernels the clustering grouped with the wrong variety.
examples = pd.DataFrame({
    "cluster": labels,
    "grouped with": [variety_names[v] for v in assigned],
    "true variety": [variety_names[v] for v in y_true],
    "margin": margin.round(2),
    "area": X["area"].to_numpy(),
    "compactness": X["compactness"].to_numpy(),
    "asymmetry": X["asymmetry"].to_numpy(),
}, index=pd.RangeIndex(len(y_true), name="kernel"))
print("\nThe disagreements, sorted from least to most certain:")
display(examples[disagree].sort_values("margin"))
print("For comparison, three kernels the clustering is most certain about:")
display(examples.loc[np.argsort(margin)[-3:]])
```

```
Kernels whose cluster majority differs from their true variety: 17 of 210
Median margin, agreeing kernels:    1.94
Median margin, disagreeing kernels: 0.41
The 15% of kernels with the smallest margin (32 kernels) contain 11 of the 17 disagreements.
```




```
The disagreements, sorted from least to most certain:
```

kernel	cluster	grouped with	true variety	margin	area	compactness	asymmetry
61	0	Canadian	Kama	0.04	11.23	0.8840	2.269
63	0	Canadian	Kama	0.06	13.22	0.8680	4.157
132	2	Kama	Rosa	0.07	15.38	0.8706	4.462
137	2	Kama	Rosa	0.12	15.57	0.8527	2.640
59	0	Canadian	Kama	0.18	12.11	0.8392	1.502
197	2	Kama	Canadian	0.32	13.37	0.8849	4.670
37	1	Rosa	Kama	0.32	17.08	0.9079	2.956
165	2	Kama	Canadian	0.40	12.10	0.8793	2.201
138	2	Kama	Rosa	0.41	15.60	0.8580	2.725
19	0	Canadian	Kama	0.44	12.72	0.8686	4.102
60	0	Canadian	Kama	0.51	11.42	0.8683	2.700
8	1	Rosa	Kama	0.79	16.63	0.8747	2.040
199	2	Kama	Canadian	1.00	12.76	0.8964	2.828
201	2	Kama	Canadian	1.01	12.67	0.8977	2.300
69	0	Canadian	Kama	1.02	12.73	0.8458	3.533
135	2	Kama	Rosa	1.37	15.38	0.8990	3.600
124	2	Kama	Rosa	1.47	15.99	0.9064	3.336

```
For comparison, three kernels the clustering is most certain about:
```

kernel	cluster	grouped with	true variety	margin	area	compactness	asymmetry
114	1	Rosa	Rosa	3.33	20.97	0.8859	4.677
77	1	Rosa	Rosa	3.33	20.71	0.8763	4.451
96	1	Rosa	Rosa	3.35	19.31	0.8815	3.477

What this shows. The disagreements are **mostly borderline cases, not random noise**:

- **They sit close to the boundaries.** Their median margin is 0.41, compared with 1.94 for the kernels that agree. 11 of the 17 fall among the 15% of kernels with the smallest margins.
- **Every disagreement involves Kama.** 8 are Kama kernels at the edges of Kama's range. Kama kernels that are small or unusually asymmetric were grouped with Canadian, for example kernels 59–61 (area 11.2–12.1) and kernel 63 (asymmetry 4.16). Large Kama kernels, numbers 8 and 37 (area 16.6–17.1), were grouped with Rosa.
- **The other 9 go the opposite way.** These are 5 Rosa and 4 Canadian kernels that look unusually Kama-like and were grouped with Kama.

On the plots, the circled points lie along the borders between the groups. For the co-op this suggests a practical rule: accept the confident assignments, and send kernels with a small margin for a manual check.

What each cluster captured

Finally, see what each cluster actually represents. The cluster centres are stored in scaled units, so turn them back into the original measurements with `scaler.inverse_transform`, and read across each row to picture the typical kernel in that group.

In [15]:
```
# Cluster centres are stored in scaled units; turn them back into original measurements.
centres = scaler.inverse_transform(km.cluster_centers_)
profiles = pd.DataFrame(centres, columns=feature_names)

profiles.index = [f"cluster {c}" for c in range(k)]
```

```
profiles.index.name = "cluster centre"
profiles["kernels"] = np.bincount(labels)
profiles["majority variety"] = [variety_names[ct.loc[c].idxmax()] for c in range(k)]

display(profiles.round(3))
```

cluster centre	area	perimeter	compactness	kernel_length	kernel_width	asymmetry	groove_length	kernels	majority variety
cluster 0	11.857	13.248	0.848	5.232	2.850	4.742	5.102	72	Canadian
cluster 1	18.495	16.203	0.884	6.176	3.698	3.632	6.042	67	Rosa
cluster 2	14.438	14.338	0.882	5.515	3.259	2.707	5.121	71	Kama

Going further: reading the profiles at a glance *(extension)*

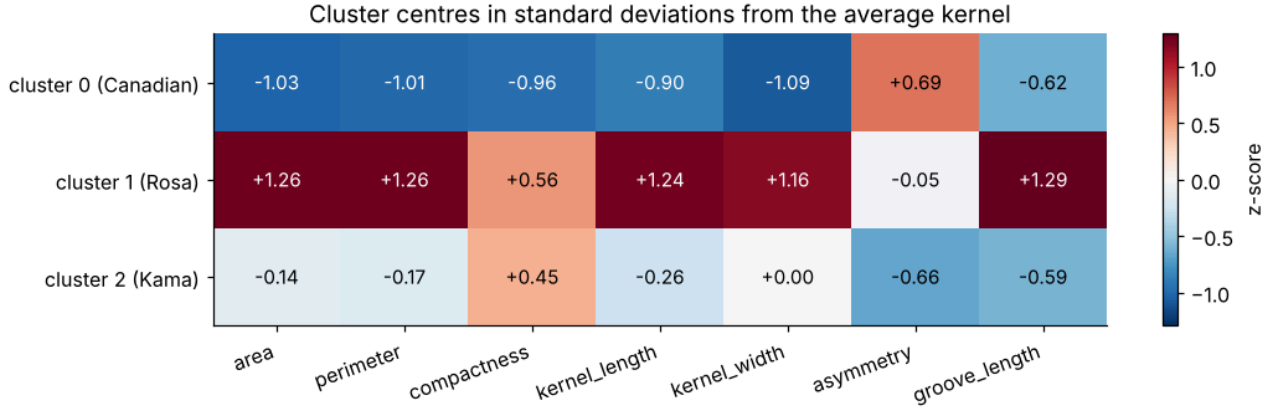
The profile table above is in original units, which are hard to compare across features: compactness varies in the third decimal place, while area varies in whole units. Showing the same centres in standard deviations from the average kernel makes each group's character easy to read. I also compare each centre with the real average kernel of its majority variety.

In [16]:

```
# Centres in standardised units (0 = average kernel, +1 = one standard deviation above).
z_centres = pd.DataFrame(km.cluster_centers_, columns=feature_names,
                         index=[f"cluster {c} ({profiles.loc[f'cluster {c}', 'majority variety']})"
                                for c in range(k)])

fig, ax = plt.subplots(figsize=(10, 3.3))
lim = np.abs(z_centres.values).max()
im = ax.imshow(z_centres.values, cmap="RdBu_r", vmin=-lim, vmax=lim, aspect="auto")
for i in range(z_centres.shape[0]):
    for j in range(z_centres.shape[1]):
        ax.text(j, i, f"{z_centres.iloc[i, j]:+.2f}", ha="center", va="center", fontsize=10,
                color="white" if abs(z_centres.iloc[i, j]) > 0.85 else "black")
ax.set_xticks(range(7), feature_names, rotation=20, ha="right")
ax.set_yticks(range(k), z_centres.index)
ax.set_title("Cluster centres in standard deviations from the average kernel")
ax.grid(False)
fig.colorbar(im, ax=ax, label="z-score")
fig.tight_layout()
fig.savefig(FIG_DIR / "09_cluster_profiles.png")
plt.show()

# How close is each cluster centre to the real average kernel of its majority variety?
variety_means = X.groupby(y_true).mean().rename(index=variety_names)
rows = []
for c in range(k):
    v = profiles.loc[f"cluster {c}", "majority variety"]
    rows.append(profiles.loc[f"cluster {c}", feature_names].rename(f"{v}: cluster {c} centre"))
    rows.append(variety_means.loc[v].rename(f"{v}: true variety mean"))
compare = pd.DataFrame(rows).astype(float)
display(compare.round(3))
```



	area	perimeter	compactness	kernel_length	kernel_width	asymmetry	groove_length
Canadian: cluster 0 centre	11.857	13.248	0.848	5.232	2.850	4.742	5.102
Canadian: true variety mean	11.874	13.248	0.849	5.230	2.854	4.788	5.116
Rosa: cluster 1 centre	18.495	16.203	0.884	6.176	3.698	3.632	6.042
Rosa: true variety mean	18.334	16.136	0.884	6.148	3.677	3.645	6.021
Kama: cluster 2 centre	14.438	14.338	0.882	5.515	3.259	2.707	5.121
Kama: true variety mean	14.334	14.294	0.880	5.508	3.245	2.667	5.087

What this shows.

- **Cluster 1 (Rosa) is the large kernel.** It sits about 1.2–1.3 standard deviations above average on all five size measurements, with an area of 18.5 against an overall average of 14.8.
- **Cluster 0 (Canadian) is small, narrow and less compact.** It also has the highest asymmetry (4.74, or +0.69 SD).
- **Cluster 2 (Kama) is a medium-sized, compact kernel.** It has low asymmetry (2.71) and a short groove.

The cluster centres sit very close to the true variety means: area 11.86 against 11.87 for Canadian, 18.50 against 18.33 for Rosa, and 14.44 against 14.33 for Kama. So the clusters describe each variety's typical kernel faithfully, even though 17 individual kernels are misplaced.

In practical terms, Rosa stands out mainly by its size, while Kama and Canadian differ in shape (compactness and asymmetry) as much as in size.

Going further: is the result specific to K-Means? *(extension)*

If the three groups are real, other clustering methods with different assumptions should find much the same partition, and K-Means should not depend on its random starting point. I compare two other methods and refit K-Means with 25 different seeds:

- **Agglomerative clustering with Ward linkage**, which builds clusters bottom-up.
- **A Gaussian mixture model**, which allows elliptical clusters with their own spread.

In [17]:
```
# Is the three-group structure specific to K-Means, or to one random start?
def agreement(name, lab):
    ct_m = pd.crosstab(lab, y_true)
    return {"method": name,
            "purity": ct_m.max(axis=1).sum() / ct_m.values.sum(),
            "ARI vs true varieties": adjusted_rand_score(y_true, lab),
            "ARI vs my K-Means clusters": adjusted_rand_score(labels, lab)}

alt = [
    agreement("K-Means (k = 3), the model used above", labels),
    agreement("Agglomerative, Ward linkage (k = 3)",
              AgglomerativeClustering(n_clusters=3, linkage="ward").fit_predict(X_scaled)),
    agreement("Gaussian mixture, full covariance (3 components)",
              GaussianMixture(n_components=3, n_init=10, random_state=RANDOM_STATE)
              .fit_predict(X_scaled)),
]
display(pd.DataFrame(alt).set_index("method").round(3))

seed_runs = [KMeans(n_clusters=3, n_init=10, random_state=s).fit_predict(X_scaled)
             for s in range(25)]
seed_ari = [adjusted_rand_score(labels, r) for r in seed_runs]
print(f"K-Means refitted with 25 different random seeds: ARI with my clustering "
      f"ranges from {min(seed_ari):.3f} to {max(seed_ari):.3f}")
```

method	purity	ARI vs true varieties	ARI vs my K-Means clusters
K-Means (k = 3), the model used above	0.919	0.773	1.000
Agglomerative, Ward linkage (k = 3)	0.929	0.797	0.836
Gaussian mixture, full covariance (3 components)	0.895	0.723	0.777

```
K-Means refitted with 25 different random seeds: ARI with my clustering ranges from 1.000 to 1.000
```

What this shows. All three methods find essentially the same three groups:

- **Ward** agrees with my K-Means clusters at ARI 0.836. It does slightly better against the varieties, with purity 0.929 and ARI 0.797.
- **The Gaussian mixture** is a little lower, with purity 0.895 and ARI 0.723.
- **K-Means** returns exactly the same partition for all 25 random seeds (ARI 1.000).

So the three-group structure is a property of the data, not of one algorithm or one lucky random start. The differences between methods are only a few percentage points of purity, which fits the picture of a clear overall structure with soft boundaries.

Going further: how close is this to the best possible? *(extension)*

A purity of 0.919 is hard to judge on its own, because 1.0 may simply not be reachable with these seven measurements. To find out what *is* reachable, I train two supervised models that **are** allowed to see the labels, and score them on kernels they did not see during training (10-fold cross-validation). Their accuracy is a realistic upper limit for how well these measurements can separate the varieties. The clustering itself is not changed.

In [18]:
```
# (a) The best these measurements allow: supervised models that are trained WITH the labels,
#     scored on held-out kernels with 10-fold cross-validation.
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)
supervised = {
    "Linear discriminant analysis (sees the labels)": LinearDiscriminantAnalysis(),
    "Logistic regression (sees the labels)": make_pipeline(StandardScaler(),
                                                           LogisticRegression(max_iter=1000)),
}
benchmark = []
for name, model in supervised.items():
    scores = cross_val_score(model, X, y_true, cv=cv)
    benchmark.append({"model": name, "share of kernels correct": scores.mean(),
                      "spread over 10 folds (std)": scores.std()})
benchmark.append({"model": "K-Means, k = 3 (never sees the labels)",
                  "share of kernels correct": (assigned == y_true).mean(),
                  "spread over 10 folds (std)": np.nan})
display(pd.DataFrame(benchmark).set_index("model").round(3))
```

model	share of kernels correct	spread over 10 folds (std)
Linear discriminant analysis (sees the labels)	0.962	0.036
Logistic regression (sees the labels)	0.933	0.053
K-Means, k = 3 (never sees the labels)	0.919	NaN

What this shows.

- **Even with the labels, the best model gets about 96% right.** Linear discriminant analysis classifies 96.2% of unseen kernels correctly (±3.6% across the folds), and logistic regression

93.3%. So 100% is not a realistic target with these seven measurements: some kernels from different varieties simply measure almost the same.
- **Clustering gets close to that without any labels.** K-Means places 91.9% of kernels with their own variety, only about 4 percentage points below the supervised best.

The comparison is not perfectly like for like: the supervised scores come from held-out kernels, while K-Means is scored on all 210. That is fair, though, because K-Means never used the labels to fit anything. It sharpens the answer to the central question: clustering recovered most of the structure that these measurements contain at all.

Going further: an idea I tested and rejected *(extension)*

In my reflection I suggested clustering on decorrelated features, because five of the seven measurements mostly repeat the same size information. Here I test that idea. The first three columns use no labels, so they are what I could have used to decide. The last two need the labels, so they can only be read after the fact.

In [19]:
```
# (b) Decorrelated ("whitened") features: PCA rotates the seven scaled features into
#     uncorrelated components and rescales each one to unit variance, so the five near-duplicate
#     size measurements no longer dominate the distances.
def describe_space(Z, name):
    sil = {kk: silhouette_score(Z, KMeans(n_clusters=kk, n_init=10,
                                          random_state=RANDOM_STATE).fit_predict(Z))
           for kk in range(2, 8)}
    lab_z = KMeans(n_clusters=3, n_init=10, random_state=RANDOM_STATE).fit_predict(Z)
    seed_agreement = [adjusted_rand_score(lab_z, KMeans(n_clusters=3, n_init=10,
                                                        random_state=s).fit_predict(Z))
                      for s in range(10)]
    ct_z = pd.crosstab(lab_z, y_true)
    return {"feature space": name,
            "silhouette at k = 3 (no labels)": sil[3],
            "k with best silhouette (no labels)": max(sil, key=sil.get),
            "lowest ARI over 10 seeds (no labels)": min(seed_agreement),
            "purity (needs labels)": ct_z.max(axis=1).sum() / ct_z.values.sum(),
            "ARI vs varieties (needs labels)": adjusted_rand_score(y_true, lab_z)}

experiment = [describe_space(X_scaled, "7 scaled features (main analysis)")]
for n_comp in (3, 4, 5, 7):
    Z = PCA(n_components=n_comp, whiten=True).fit_transform(X_scaled)
    experiment.append(describe_space(Z, f"{n_comp} whitened components"))
display(pd.DataFrame(experiment).set_index("feature space").round(3))
```

feature space	silhouette at k = 3 (no labels)	k with best silhouette (no labels)	lowest ARI over 10 seeds (no labels)	purity (needs labels)	ARI vs varieties (needs labels)
7 scaled features (main analysis)	0.401	2	1.000	0.919	0.773
3 whitened components	0.326	3	1.000	0.890	0.708
4 whitened components	0.270	3	0.931	0.957	0.876
5 whitened components	0.229	6	0.944	0.952	0.865
7 whitened components	0.164	5	0.583	0.957	0.876

What this shows. On paper, decorrelating the features helps: with 4 or 7 whitened components, purity rises to 0.957 and ARI to 0.876. I did not adopt it, for three reasons.

- **Nothing without labels points to the good versions.** Judged only by the label-free columns, the most convincing whitened version has 3 components: its silhouette is the highest of the four, it

peaks at k = 3, and it gives the identical answer for every seed. Yet it scores *worse* than my main analysis (purity 0.890). The versions that score well can only be recognised with the answer key.
- **Some versions are unstable or point to a different k.** With all 7 components the clustering changes with the random start (ARI as low as 0.583), and the silhouette prefers k = 5. With 5 components it prefers k = 6.
- **Choosing by purity would break the project's rule.** Picking the feature space because it matches the hidden labels better would be tuning the method to the answer key, and the higher score would no longer measure what clustering finds on its own.

Silhouette values are not directly comparable between different feature spaces, so I compare the versions mainly on whether their own diagnostics point to k = 3 and whether they are stable. My main analysis therefore stays as it is. The experiment still points to useful future work: with a small labelled sample, the feature space could be chosen fairly.

Ethical considerations

K-Means always returns exactly k groups, even from data with no real structure, so these clusters are a *hypothesis* about the varieties, not proof of them. If the co-op treated them as definite varieties, about 8% of kernels (17 of 210) would be mislabelled. Every one of those errors involves Kama, so any pricing, sorting or quality decision built on the clusters would be systematically less reliable for one variety.

The purity of 0.919 only means something together with k = 3, because purity can be pushed to 1.0 just by making more clusters. The choice of k was itself a judgement between an elbow at 3 and a silhouette at 2. So the honest claim is that *the measurements are consistent with about three varieties, which can be separated for roughly 11 kernels in 12*, and that claim should be confirmed on a labelled sample before acting on it.

The data also come from a single set of experimental fields, so a new harvest or a different variety mix could shift the boundaries or add groups this analysis cannot see.

Wrapping up

The questions this section answers:

- How many groups did the diagnostics support, and how did you decide when the elbow and the silhouette disagreed?
- From the cross-tabulation and the purity score, how much of the real variety structure did the clustering recover?
- Which variety was separated most cleanly, and which two were the hardest to tell apart? Does the coloured scatter support this interpretation?
- Why must you be cautious when judging a seven-dimensional clustering from a two-dimensional PCA plot?

My answer. Without ever using the labels, clustering recovered most of the real variety structure.

Choosing the number of groups. The diagnostics disagreed: the elbow pointed to k = 3 and the silhouette to k = 2 (0.466 against 0.401). I chose 3 because the elbow was sharp and the k = 3 silhouette was a close second. Two extra label-free checks backed this up: the gap statistic selected 3, and bootstrap stability showed that three clusters are about as reproducible as two, while k ≥ 4 is unstable.

How much was recovered. Once the varieties were revealed, every cluster was dominated by one variety. **Purity is 0.919** (193 of 210 kernels), with **ARI 0.773** and **NMI 0.728**, all far above chance. For context, a supervised model trained *with* the labels gets 96.2% of unseen kernels right, so clustering recovered most of the structure that these measurements contain at all.

Cleanest and hardest varieties. Rosa was separated most cleanly: its cluster is 97% Rosa, and it was never mixed with Canadian. Kama sits between the other two and was the hard case. **Kama and Canadian** were the hardest pair to tell apart (10 kernels swapped), followed by Kama and Rosa (7). The coloured scatter supports this, with Canadian on the left, Rosa on the right, and Kama in the middle overlapping both.

Reading the PCA plot with caution. The two components keep 89% of the variance, so 11% of the variation is not shown, and K-Means measured distances in all seven dimensions. Points that overlap in the plot may be separated in the hidden dimensions, and points that look safely inside one colour may be closer to another centre in seven dimensions. That is why my conclusions rest on the cross-tabulation and the scores rather than on the picture.

In short: the three-variety structure is real and can largely be recovered from size and shape measurements alone. Its boundaries are soft, though, and about one kernel in twelve would need another way to be identified.

Reflection

What worked well.

- **Keeping the labels hidden until Part D** made the validation honest: k was fixed before I saw any variety.
- **Using several independent, label-free diagnostics** turned a confusing disagreement (elbow against silhouette) into a reasoned decision instead of a guess.
- **The profiles were easy to interpret.** Once converted back to real units and z-scores, the clusters mapped onto clear physical descriptions (large, medium-compact, small-asymmetric), and their centres matched the true variety means almost exactly.

What was difficult.

- **There is no single correct k**, and the standard tools disagreed, so I had to weigh evidence instead of reading off an answer.
- **It was hard not to over-read the 2-D PCA plot.** It looks persuasive, but it hides 11% of the variation, and the clustering works in seven dimensions.
- **Purity is easy to misuse.** Purity alone would have looked better with more clusters, which is why I added ARI and NMI.
- **Resisting a better-looking score.** Decorrelated features raised purity to 0.957, but only in versions I could have picked by looking at the labels, so I kept my original analysis.

What I would improve with more time.

1. **Choosing the features fairly.** I tested decorrelated features (the last extension). They can score higher, but nothing without labels tells you which version to use. With more time I would look for a label-free way to choose the feature space, or use a small labelled sample to choose it honestly.
2. **Soft assignments.** A Gaussian mixture gives each kernel a probability for each group. That would let the co-op mark uncertain kernels for manual checking instead of forcing every kernel into a group.

3. **More and newer data.** With kernels from other harvests and fields, I could test whether the three groups and their boundaries hold up outside this one sample.
4. **Model-based choice of k.** Information criteria such as BIC for mixture models would be a useful extra check, alongside the gap statistic.

Reproducibility

The library versions this notebook was last run with. Every random step uses `random_state=42` (or a seed derived from it), so a fresh top-to-bottom run reproduces every number above.

In [20]:
```
import sys, sklearn, matplotlib, scipy
print("Python      ", sys.version.split()[0])
for lib in (np, pd, sklearn, scipy, matplotlib):
    print(f"{lib.__name__:<12}", lib.__version__)
```

```
Python       3.13.16
numpy        2.5.3
pandas       3.0.5
sklearn      1.9.1
scipy        1.18.1
matplotlib   3.11.2
```